



華南師範大學

本科学生实验（实践）报告

院 系：计算机学院

实验课程：编译原理实验

实验项目：实验 3：TINY 扩充语言的语法树生成

指导老师：黄煜廉

开课时间：2025 ~ 2026 年度第 1 学期

专 业：计算机科学与技术

班 级：计科 2 班

学生姓名：袁知本

华南师范大学教务处

华南师范大学实验报告

学生姓名 袁知本 学 号 20232131006
专 业 计算机科学与技术 年级、班级 2023 级 计科 2 班
课程名称 编译原理实验 实验项目 实验 3: TINY 扩充语言的语法树生成
实验类型 ☐验证 ☐设计 ☐综合 实验时间 2025 年 5 月 7 日
实验指导老师 黄煜廉 实验评分 _____

一、实验内容

必做内容:

(一) 为 Tiny 语言扩充的语法

1. 实现改写书写格式的新 if 语句;
2. 扩充算术表达式的运算符: ++ (前置自增 1)、-- (前置自减 1) 运算符 (类似于 C 语言的++和--运算符, 但不需要完成后置的自增 1 和自减 1)、求余%、乘方^;
3. 扩充扩充比较运算符: <=(小于等于)、>(大于)、>=(大于等于)、<>(不等于)等比较运算符;
4. 增加正则表达式, 其支持的运算符有: 或(|)、连接(&)、闭包(#)、括号()、可选运算符(?) 和基本正则表达式;
5. for 语句的语法规则 (类似于 C 语言的 for 语言格式):
书写格式: for(循环变量赋初值;条件;循环变量自增或自减 1) 语句序列
6. while 语句的语法规则 (类似于 C 语言的 while 语言格式): 书写格式: while(条件) 语句序列 endwhile

(二) 对应的语法规则

1. 把 TINY 语言原有的 if 语句书写格式
if_stmt-->if exp then stmt-sequence end | if exp then
stmt-sequence else stmt-sequence end

改写为：

```
if_stmt-->if(exp) stmt-sequence else stmt-sequence |  
if(exp) stmt-sequence
```

2. ++（前置自增 1）、--（前置自减 1）运算符、求余%、乘方^等运算符的文法规则请自行组织
3. <=(小于等于)、>(大于)、>=(大于等于)、<>(不等于)等运算符的文法规则请自行组织
4. 为 tiny 语言增加一种新的表达式——正则表达式，其支持的运算符有： 或(|) 、连接(&)、闭包(#)、括号() 、可选运算符(?) 和基本正则表达式，对应的文法规则请自行组织
5. 为 tiny 语言增加一种新的语句，ID::=正则表达式 （同时增加正则表达式的赋值运算符::=）
6. 为 tiny 语言增加一个符合上述 for 循环的书写格式的文法规则
7. 为 tiny 语言增加一个符合上述 while 循环的书写格式的文法规则
8. 为了实现以上的扩充或改写功能，还需要注意对原 tiny 语言的文法规则做一些相应的改造处理

（三）实验要求

- （1）要提供一个源程序编辑的界面，以让用户输入源程序（可输入，可保存、可打开源程序）
- （2）可由用户选择是否生成语法树，并可查看所生成的语法树
- （3）实验 3 的实现只能选用的程序设计语言为：C++
- （4）要求应用程序的操作界面应为 Windows 界面

选做内容：

二、实验目的

1. 实现一个带图形化界面的 TINY 扩充语言的语法树生成程序
2. 用户可以通过直接输入、拖拽文件、通过文件对话框等方式输入 TINY 源程序，可由用户选择生成语法树，并查看语法规则匹配结果
3. 深入了解语法树生成的基本原理和实现方法，同时掌握通过 QT 开发图形化界面项目的要点
4. 深入了解 TINY 语言的文法规则，能够在理解原有文法的基础上进行文法的改造和添加处理

三、实验文档：

注：本次实验中的必做内容均已完成，同时也完成了全部的选做内容

1. 实验需求分析

本实验的目标是用 C++ 语言实现一个具有图形化界面的 TINY 扩充语言的语法树生成系统，要为用户实现文件拖拽、文件对话框选择和直接输入等多种输入 TINY 源代码的方式，需实现由用户选择生成语法树的功能和源代码文件保存功能，并且在生成语法树时要有源码匹配结果状态的显示，同时，通过 QT 平台构建一个用户友好型可视化图形界面

功能需求

1) 用户界面设计

用户界面包括状态信息栏、TINY 源代码输入区、语法分析（语法树生成）区共 3 个模块：

- 状态信息栏：显示提示信息，显示用户输入的 TINY 源代码的语法规则匹配结果，若有错误，则会指出在哪一行的哪一个 token 出错；
- TINY 源代码输入区：可以通过直接输入 TINY 源代码、拖拽文件和打开文件对话框等三个方式将 TINY 源代码输入到输入区中，点击【保存源程序】可以进行保存，点击

【读取源程序文件】可以实现打开文件对话框选择文件

- 语法分析（语法树生成）区：借助 Qt 中的 `treeWidget` 控件，实现生成语法树后的输出展示，该功能由用户点击按钮【生成语法树】实现

2) TINY 语言文法规则的扩充与改写

本次实验的程序基于原有的 TINY 语言文法规则，在其基础上做了部分修改和语句扩充，具体如下：

1. if 语句的改写；
2. for 语句、while 语句的新增与改写（C 语言风格）；
3. 运算符（自增++、自减--、求余%、乘方^等）、比较符号（小于等于<=、大于>、不等于<>、大于等于>=等）、正则表达式运算符（或|、连接&、闭包#、括号()、可选?和基本正则表达式）的新增；
4. 新增语句 `ID::=正则表达式`（同时新增正则表达式的赋值运算符`::=`）

3) 生成语法树

基于原有的 TINY 语言文法规则，加上扩充的内容以及改写的部分，通过 `match` 函数实现读取 token 后分类，构建语法树，将构建完成的语法树返回，通过 Qt 中的 `treeWidget` 控件，实现生成语法树后的输出展示

2. 实验环境

硬件设备：基于 x64 的处理器，RAM16.0GB

软件设备：Windows11 家庭版，

Qt5.14.2，

Qt Creator 4.11.1(Community)

3. 系统设计

- 1) 简洁方便的交互界面：

用 Qt 开发的可视化交互界面如下图所示：

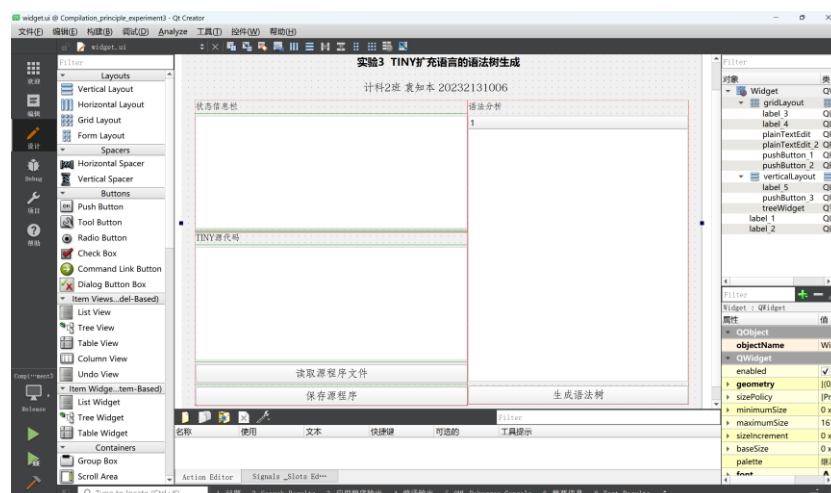


图 3-1-1 交互界面

2) TINY 语言语法规则的扩充与改写：

本功能包含以下部分：

- **if 语句文法的改写：**根据实验三要求，把 TINY 语言原有的 if 语句书写格式

if_stmt-->if exp then stmt-sequence end | if exp then
stmt-sequence else stmt-sequence end

改写为：

if_stmt-->if(exp) stmt-sequence else stmt-sequence | if(exp)
stmt-sequence

由于在 TINY 语言中，大括号已经被用于注释，而若无括号包裹 if 语句下的代码块，则容易出现 else 挂载位置匹配错误的问题，也即二义性问题，基于此我对 if 语句的文法规则做出相应的修改，改为用中括号[]包裹 if 语句下的代码块，以此区分 if 语句下代码块的范围，因此 if 语句的文法规则改写为如下所示：

if_stmt->if(exp) [[stmt-sequence] else [stmt-sequence]]

修改后，原先的 if 语句中的 then、end 将弃用

- **for 语句文法改写：**

根据实验三要求，把 TINY 语言原有的 for 语句的文法规则修改为如下：

`for_stmt -> for(assign_stmt;exp;assign_stmt) stmt_sequence`

由于要修改为 C 风格的 for 语句，故原有的 for 语句文法规则中的 `downto`、`to` 都将弃用

- **while 语句文法扩充：**

在原本的 TINY 语言文法规则中，不存在 while 语句，故我们需要扩充该语句的文法到 TINY 语言中：

`while_stmt -> while(exp) stmt_sequence endwhile`

通过 `endwhile` 作为来判断 while 语句的结束

- **运算符的新增：**本次实验中新增了++（前置自增 1）、--（前置自减 1）运算符、求余%、乘方^等运算符，其中实验要求的++和--运算我都做了额外的后置版本扩充，使得扩充后的 TINY 语言功能更加强大，而根据 C++中运算符的优先级顺序，我们可知 TINY 语言中运算符的优先级顺序（从上到下为由高到低）如下：

1. ++ --

2. ^

3. * / %

4. + -

5. +=

因此，根据上述的优先级顺序，我们可以制定文法规则，转换成 EBNF 后如下：

`plusplus -> ++`

`subsub -> --`

`addop -> + | -`

`power -> factor {powop factor}`

powop -> ^

factor -> (exp) | number | identifier

mulop -> * | / | %

- **比较运算符新增：**本次实验中新增了<=(小于等于)、>(大于)、>=(大于等于)、<>(不等于)等比较运算符，与运算符同理，我们也可以写出其语法规则，转 EBNF 后如下：

andop -> and

andexp -> simple-exp [comparison-op simple-exp]

orop -> or

orex -> andexp {andop andexp}

exp -> orexp {orop orexp}

notop -> not

notexp -> {notop} power

simple-exp -> simple-exp addop term | term

term -> notexp {mulop notexp}

- **正则表达式运算符新增：**正则表达式与算术表达式的操作不同，前者是无需计算的，因此不允许出现算术表达式和正则表达式杂糅的情况，因此，实验三的要求中引入了“ID::=正则表达式”这一语句，用::=表示正则表达式的赋值，和:=表示的正常赋值区分开来

在之前的实验二中，我们已经明确了正则表达式运算符的优先级（从上到下优先级由高到低）：

1. 闭包 #可选 ?
2. 连接 &
3. 或 |
4. 括号 ()

因此，我们可以得到下面的转 EBNF 后的文法规则：

```
regex_stmt -> andreg{orop andreg}
orop -> |
andreg -> topreg{andop topreg}
andop -> &
topreg -> reg_factor{topop}
topop -> # | ?
reg_factor -> (regex_stmt) | identifier | number
```

3) 生成语法树:

基于原有的 TINY 语言文法规则，加上扩充的内容以及改写的部分，通过 `match` 函数实现读取 `token` 后分类，构建语法树，将构建完成的语法树返回，通过 Qt 中的 `treeWidget` 控件，实现生成语法树后的输出展示

4. 系统实现

4.1 用户界面设计

4.1.1 布局:

主窗口包含状态信息栏、TINY 源代码输入栏和语法分析显示窗口，TINY 源代码输入框支持拖拽文件、通过文件对话框选取以及直接输入三种方式导入 TINY 源代码；状态信息栏显示通过语法规则匹配 TINY 源代码的结果，若匹配出错则会指出发生错误的对应的行和位置；语法分析显示窗口能够显示构建的语法树；按钮【读取 TINY 源程序】实现通过文件对话框选取 TINY 源代码文件；按钮【保存源程序】实现将当前输入在输入框中的 TINY 源程序保存下来；按钮【生成语法树】实现借助 Qt 中的 `treeWidget` 控件输出显示语法树
主窗口界面如下图所示：



图 4-1-1 主窗口控件布局

4.1.2 交互逻辑：

点击按钮【读取源程序文件】，即可打开文件对话框，在文件对话框中选择你要导入的 TINY 源代码文件，单击【确定】即可导入示例如下：

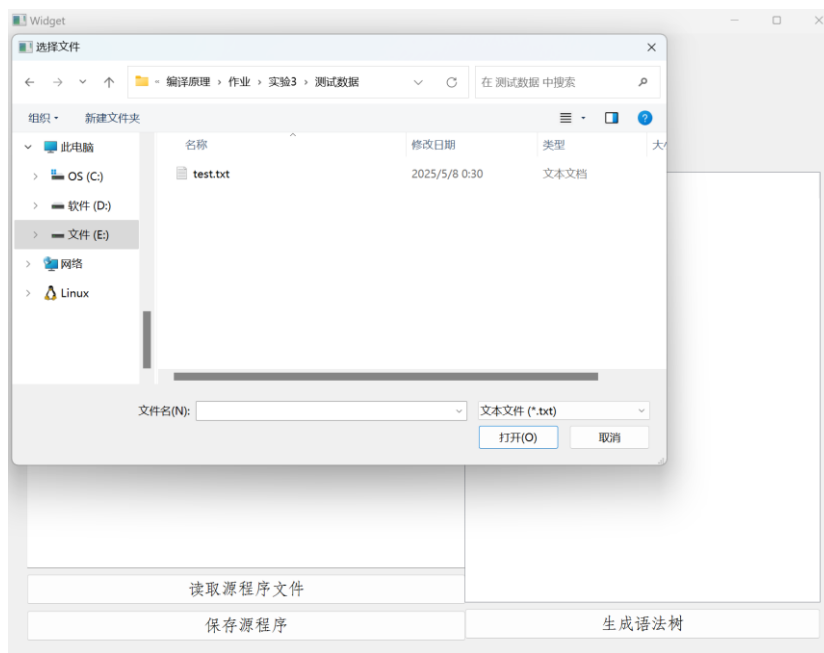




图 4-1-2、图 4-1-3 TINY 源代码导入功能展示

如上图所示，除了直接输入、文件拖拽的方式导入 TINY 源代码，也可以通过点击【读取源程序文件】实现通过打开文件对话框，选择本地的 TINY 源程序导入

在导入或手动输入了 TINY 源代码后，点击按钮【保存源程序】，即可打开文件对话框，自定义保存的文件名、保存的位置路径，点击【保存】即可将当前输入到 TINY 源代码输入框中的 TINY 源代码保存到你刚才指定的文件和位置中，其展示分别如下：

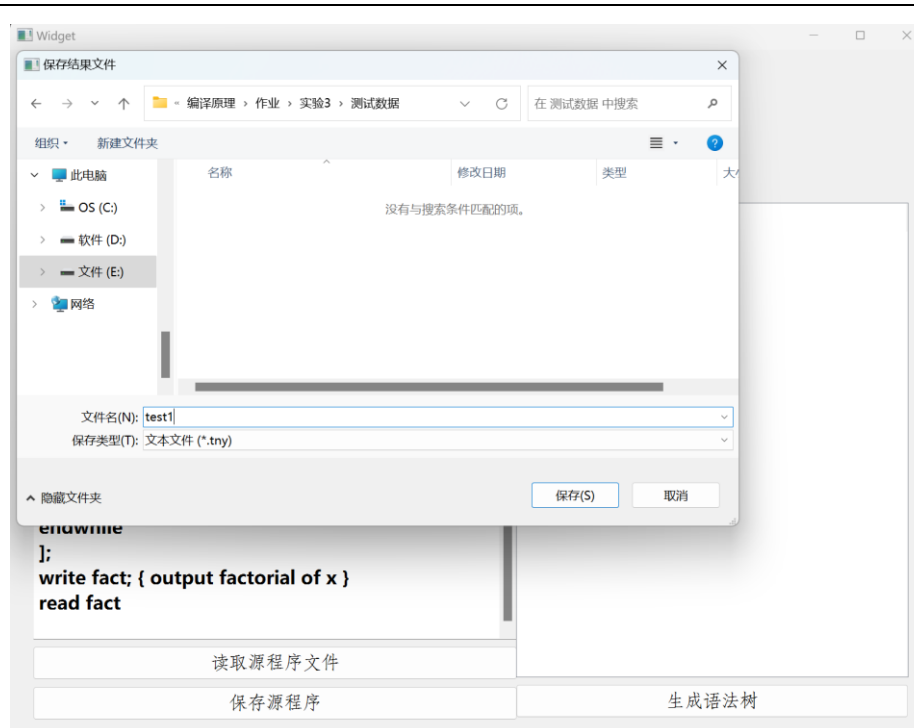


图 4-1-4 保存源代码功能演示（打开文件对话框）

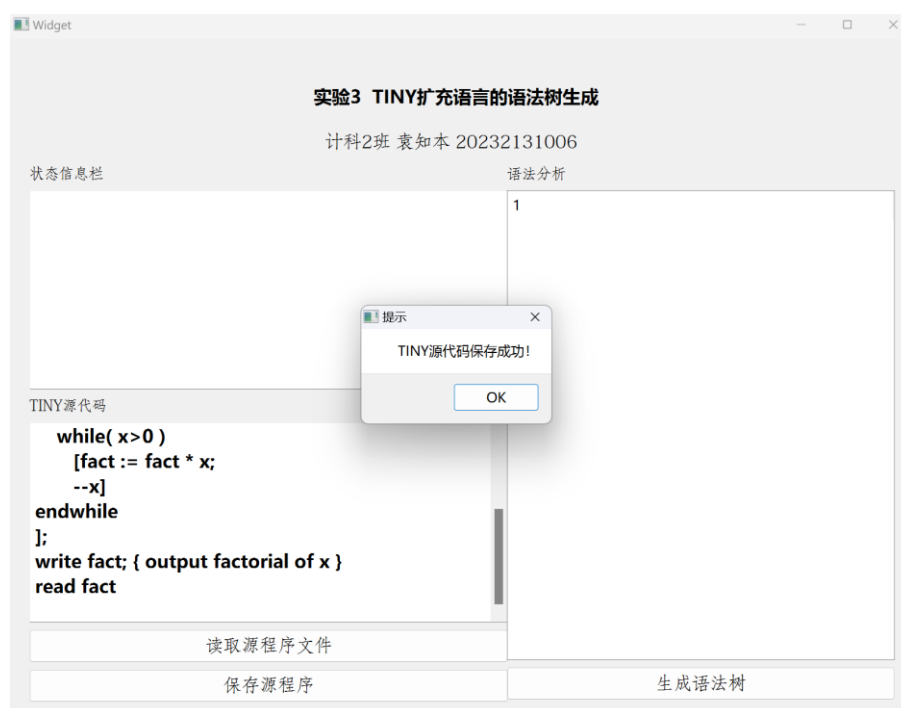


图 4-1-5 保存源代码功能演示（保存成功的提示）

在导入或手动输入了 TINY 源代码后，点击按钮【生成语法树】，系统将会将这段 TINY 源代码写入到一个临时文件 tempfile 中，然后开始根据修改与扩充后的 TINY 语言语法规则，对你输入的 TINY 源代码进行匹配，并构建出对应的语法树，最后构造完成后将语法树借助 Qt 中的 treeWidget 控件输出显示，同时，状态信息栏中也会显示匹配

的结果，若有错误，则会指出错误的位置，具体到某一行中的出错的 token 位置，运行效果如下：

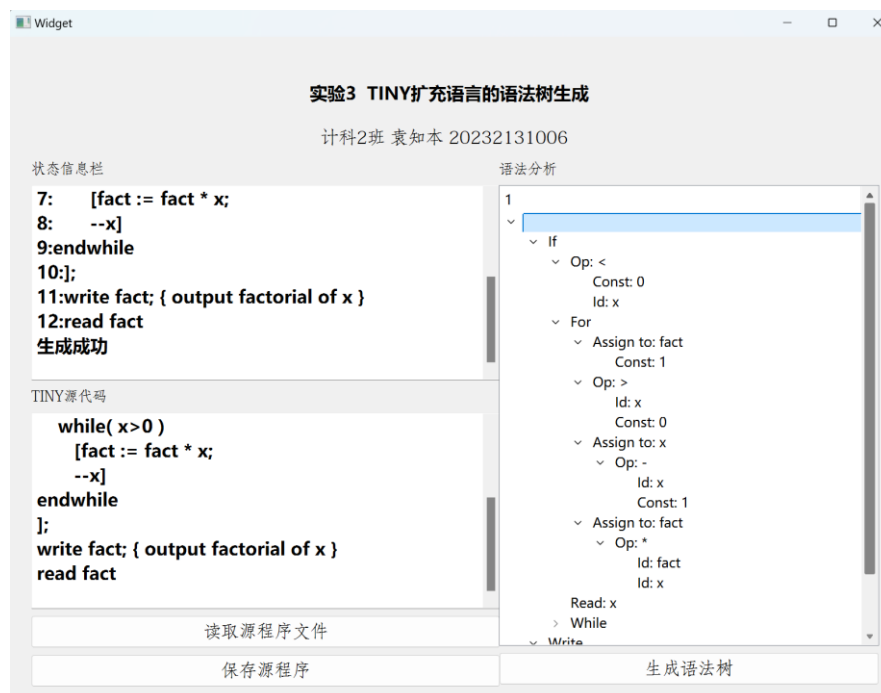
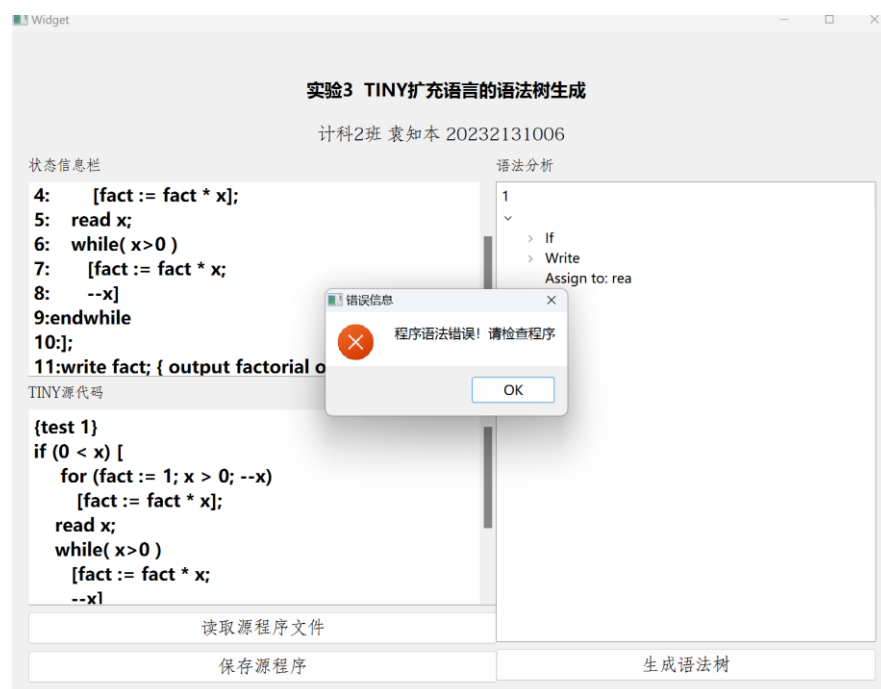


图 4-1-6 语法树构建与展示功能演示



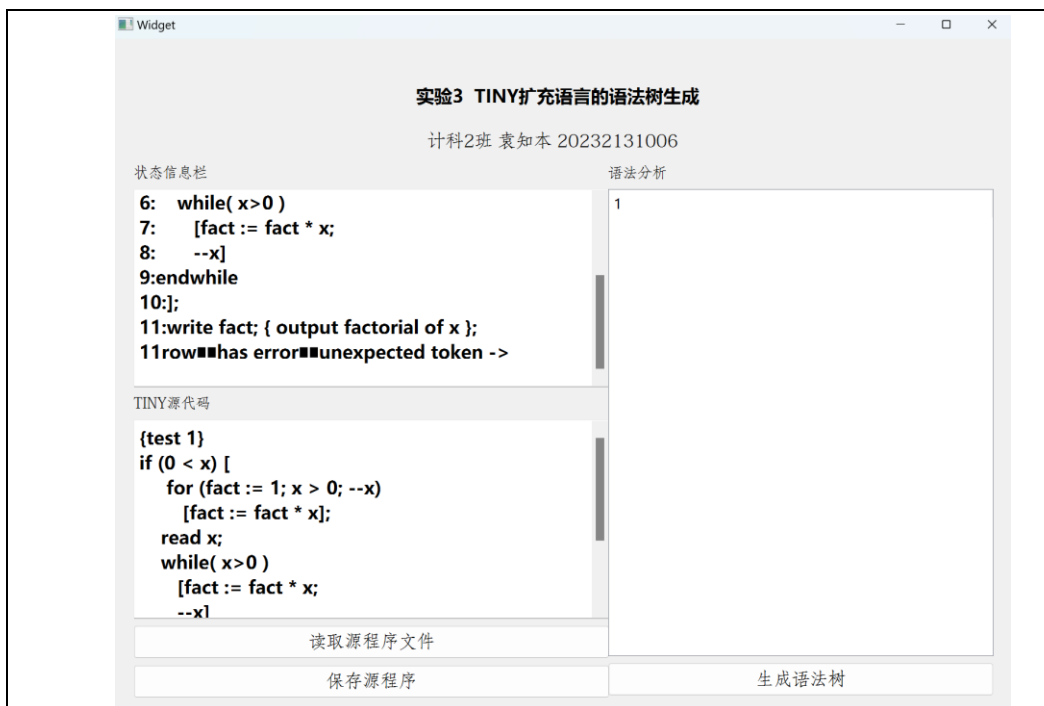


图 4-1-7、图 4-1-8 语法树构建与展示功能演示

图 4-1-8 中指出匹配出错的代码位置处，由于编码问题没有正确显示部分字符，但指出的报错位置是正确的，功能没有问题

4. 1. 3 用户交互界面的实现：

控件的实现：

如图 4-1-9 所示，用户交互界面的有按钮、文本编辑框和树型控件等若干控件，均可通过 Qt 的工具栏拖拽进行添加，然后给文本编辑框和树型控件等控件设置对应的变量，对应的按钮设置响应函数，其代码如下图所示：

```

1  #ifndef WIDGET_H
2  #define WIDGET_H
3
4  #include <QWidget>
5
6  QT_BEGIN_NAMESPACE
7  namespace Ui { class Widget; }
8  QT_END_NAMESPACE
9
10 ▼ class Widget : public QWidget
11     {
12     public:
13         Q_OBJECT
14
15     public:
16         Widget(QWidget *parent = nullptr);
17         ~Widget();
18
19     private slots:
20         // 读取源程序的按钮控件
21         void on_pushButton_1_clicked();
22         // 保存源程序的按钮控件
23         void on_pushButton_2_clicked();
24         // 生成语法树的按钮控件
25         void on_pushButton_3_clicked();
26
27     private:
28         Ui::Widget *ui;
29     };
30 #endif // WIDGET_H

```

图 4-1-9 响应函数一览

```

25 ▼ class Ui_Widget
26 {
27     public:
28         QLabel *label_1;
29         QLabel *label_2;
30         QWidget *widget;
31         QGridLayout *gridLayout;
32         QLineEdit *plainTextEdit_2;
33         QLabel *label_3;
34         QLineEdit *plainTextEdit;
35         QLabel *label_4;
36         QPushButton *pushButton_2;
37         QPushButton *pushButton_1;
38         QWidget *widget1;
39         QVBoxLayout *verticalLayout;
40         QLabel *label_5;
41         QTreeWidget *treeWidget;
42         QPushButton *pushButton_3;

```

图 4-1-10 控件变量一览

4.2 功能实现

4-2-1 修改 if 语句文法规则匹配实现:

if 语句的文法规则改写为如下所示:

if_stmt->if(exp) [[stmt-sequence] else [stmt-sequence]]

实现 if 语句文法规则匹配的代码如下:

```

109  ▼  TreeNode* if_stmt(void)
110  {
111      TreeNode* t = newStmtNode(IfK);
112      match(IF);
113      match(LPAREN);
114      if (t) t->child[0] = exp();
115      match(RPAREN);
116      match(LPM);
117      if (t) t->child[1] = stmt_sequence();
118      match(RPM);
119      ▼  if (token == ELSE) {
120          match(ELSE);
121          match(LPM);
122          if (t) t->child[2] = stmt_sequence();
123          match(RPM);
124      }
125      return t;
126  }

```

图 4-2-1 if 语句文法规则匹配函数

4-2-2 for、while 语句文法规则匹配实现：

根据实验三要求，把 TINY 语言原有的 for 语句的文法规则修改为如下：

for_stmt -> for(assign_stmt;exp;assign_stmt) stmt_sequence

据此，实现 for 语句文法规则匹配的代码如下图所示：

```

474  // 修改后的for_stmt
475  // for_stmt -> for(assign_stmt;exp;assign_stmt) stmt_sequence
476  ▼  TreeNode* for_stmt(void)
477  {
478      TreeNode* t = newStmtNode(ForK);
479      match(FOR);
480      match(LPAREN);
481      // 处理初始化赋值部分
482      if (t) t->child[0] = assign_stmt();
483      match(SEMI);
484      // 处理条件表达式部分
485      if (t) t->child[1] = exp();
486      match(SEMI);
487      // 处理更新赋值部分
488      ▼  if (t)
489      {
490          if (token == PLUSPLUS) t->child[2] = plusplus_stmt();
491          else if (token == SUBSUB) t->child[2] = subsub_stmt();
492          else t->child[2] = assign_stmt();
493      }
494      match(RPAREN);
495      match(LPM);
496      if (t) t->child[3] = stmt_sequence();
497      match(RPM);
498      return t;
499  }

```

图 4-2-2 for 语句文法规则匹配函数

在原本的 TINY 语言文法规则中，不存在 while 语句，故我们需要扩充该语句的文法到 TINY 语言中：

while_stmt -> while(exp) stmt_sequence endwhile

通过 endwhile 作为来判断 while 语句的结束

实现 while 语句文法规则匹配的代码如下图所示：

```
501 // 新增while_stmt
502 // while_stmt -> while(exp) stmt_sequence endwhile
503 ▼ TreeNode* while_stmt(void)
504 {
505     TreeNode* t = newStmtNode(WhileK);
506     match(WHILE);
507     match(LPAREN);
508     if (t) t->child[0] = exp();
509     match(RPAREN);
510     ▼ if(token == LPM)
511     {
512         match(LPM);
513         if (t) t->child[1] = stmt_sequence();
514         match(RPM);
515     }
516     ▼ else
517     {
518         TreeNode* s = statement();
519         if (t) t->child[1] = s;
520     }
521     match(ENDWHILE);
522     return t;
523 }
```

图 4-2-3 while 语句文法规则匹配函数

4-2-3 正则表达式语句文法规则匹配实现：

根据之前分析出来的正则表达式文法，我们可以给出如下的正则表达式的语句文法规则匹配函数：

```

525 ▼ TreeNode* regex_stmt(void)
526 {
527     TreeNode* t = andreg();
528 ▼ while ((token == RGOR)) ⚡ Δ equality
529 {
530     TreeNode* p = newExpNode(OpK);
531 ▼ if (p != NULL) {
532         p->child[0] = t;
533         p->attr.op = token;
534         t = p;
535         match(token);
536         t->child[1] = andreg();
537     }
538 }
539 return t;
540 }

```

图 4-2-4 regex_stmt 文法规则匹配函数

```

575 ▼ TreeNode* reg_factor(void)
576 {
577     TreeNode* t = NULL;
578 ▼ switch (token) {
579     case NUM:
580         t = newExpNode(ConstK);
581         if ((t) && (token == NUM))
582             t->attr.val = atoi(tokenString);
583         match(NUM);
584         break;
585     case ID:
586         t = newExpNode(IdK);
587         if ((t) && (token == ID))
588             t->attr.name = copyString(tokenString);
589         match(ID);
590         break;
591     case LPAREN:
592         match(LPAREN);
593         t = regex_stmt();
594         match(RPAREN);
595         break;
596     default:
597         syntaxError("unexpected token -> ");
598         printToken(token, tokenString);
599         token = getToken();
600         break;
601 }
602 return t;
603 }

```

图 4-2-5 reg_factor 文法规则匹配函数

```

222         case '|':
223             currentToken = RGOR;
224             break;
225         case '&':
226             currentToken = RGAND;
227             break;
228         case '#':
229             currentToken = RGCLOSE;
230             break;
231         case '?':
232             currentToken = RGCHOOSE;
233             break;

```

图 4-2-6 正则表达式运算符匹配函数

4-2-4 新增运算符和比较符号的匹配实现:

根据之前分析出来的新增的运算符和比较符号文法，我们可以给出如下的文法规则匹配函数：

```

195         case '*':
196             currentToken = TIMES;
197             break;
198         case '/':
199             currentToken = OVER;
200             break;
201         case '(':
202             currentToken = LPAREN;
203             break;
204         case ')':
205             currentToken = RPAREN;
206             break;
207         case '[':
208             currentToken = LPM;
209             break;
210         case ']':
211             currentToken = RPM;
212             break;
213         case ';':
214             currentToken = SEMI;
215             break;
216         case '%':
217             currentToken = MOD;
218             break;
219         case '^':
220             currentToken = POWER;
221             break;

```

图 4-2-7 新增运算符匹配

```

167         case '+':
168             c = getNextChar(); // Read the next character
169             if (c == '=')
170             {
171                 currentToken = PLUSEQ; // Token for +=
172             }
173             else if (c == '+')
174             {
175                 currentToken = PLUSPLUS; // Token for ++
176             }
177             else
178             {
179                 ungetNextChar(); // Backup if it's not += and ++
180                 currentToken = PLUS; // Token for +
181             }
182             break;
183         case '-':
184             c = getNextChar(); // Read the next character
185             if (c == '-')
186             {
187                 currentToken = SUBSUB; // Token for --
188             }
189             else
190             {
191                 ungetNextChar(); // Backup if it's not --
192                 currentToken = MINUS; // Token for -
193             }
194             break;

```

图 4-2-9 新增多元运算符符号匹配

```

128     state = DONE;
129     switch (c)
130     {
131     case EOF:
132         save = FALSE;
133         currentToken = ENDFILE;
134         break;
135     case '=':
136         currentToken = EQ;
137         break;
138     case '>':
139         c = getNextChar(); // Read the next character
140         if (c == '=')
141         {
142             currentToken = RTEQ; // Token for >=
143         }
144         else
145         {
146             ungetNextChar(); // Backup if it's not >= or <>
147             currentToken = RT; // Token for >
148         }
149         break;
150     case '<':
151         c = getNextChar(); // Read the next character
152         if (c == '=')
153         {
154             currentToken = LTEQ; // Token for <=
155         }
156         else if (c == '>')
157         {
158             currentToken = NOTEQ; // Token for <>
159         }
160         else
161         {
162             ungetNextChar(); // Backup if it's not <=
163             currentToken = LT; // Token for <
164         }
165         break;

```

图 4-2-9 新增多元比较符号匹配

```

427 // 新增乘方运算
428 TreeNode* power(void)
429 {
430     TreeNode* t = factor();
431     while ((token == POWER))
432     {
433         TreeNode* p = newExpNode(OpK);
434         if (p != NULL) {
435             p->child[0] = t;
436             p->attr.op = token;
437             t = p;
438             match(token);
439             p->child[1] = factor();
440         }
441     }
442     return t;
443 }

```

图 4-2-10 新增乘方运算符匹配

```

181 // 新增判断前置自增的文法 (plusplus)
182 TreeNode* plusplus_stmt(void)
183 {
184     // 创建一个赋值语句节点, 类型为 AssignK
185     TreeNode* t = newStmtNode(AssignK);
186     if ((t) && (token == PLUSPLUS)) {
187         // 保存变量名
188         match(PLUSPLUS); // 消耗掉前置 ++ 符号
189         if (t && token == ID) {
190             // 记录自增操作的目标变量名
191             t->attr.name = copyString(tokenString);
192             // 构造表达式部分: 等价于 x = x + 1
193             TreeNode* p = newExpNode(OpK);
194             if (p != NULL) {
195                 // 左子节点: 标识符节点, 表示原始变量 x
196                 TreeNode* left = newExpNode(IdK);
197                 if (left) {
198                     left->attr.name = copyString(t->attr.name);
199                 }
200                 p->child[0] = left;
201
202                 // 操作符: PLUS
203                 p->attr.op = PLUS;
204
205                 // 右子节点: 常量节点, 表示字面量 1
206                 TreeNode* right = newExpNode(ConstK);
207                 if (right) {
208                     right->attr.val = 1;
209                 }
210                 p->child[1] = right;
211
212                 // 将整个加法表达式作为赋值语句的右子节点
213                 t->child[0] = p;
214             }
215         }
216         // 消耗掉标识符 token
217         match(ID);
218     }
219     return t;
220 }

```

图 4-2-11 新增自增符号运算文法规则匹配函数(同时实现了前置和后置自增)

```

140 // 新增判断前置自减的文法(subsub)
141 TreeNode* subsub_stmt(void)
142 {
143     // 创建一个赋值语句节点, 类型为 AssignK
144     TreeNode* t = newStmtNode(AssignK);
145     if ((t) && (token == SUBSUB)) {
146         // 保存变量名
147         match(SUBSUB); // 消耗掉前置 -- 符号
148         if (t && token == ID) {
149             // 记录自增操作的目标变量名
150             t->attr.name = copyString(tokenString);
151             // 构造表达式部分: 等价于 x = x - 1
152             TreeNode* p = newExpNode(OpK);
153             if (p != NULL) {
154                 // 左子节点: 标识符节点, 表示原始变量 x
155                 TreeNode* left = newExpNode(IdK);
156                 if (left) {
157                     left->attr.name = copyString(t->attr.name);
158                 }
159                 p->child[0] = left;
160
161                 // 操作符: MINUS
162                 p->attr.op = MINUS;
163
164                 // 右子节点: 常量节点, 表示字面量 1
165                 TreeNode* right = newExpNode(ConstK);
166                 if (right) {
167                     right->attr.val = 1;
168                 }
169                 p->child[1] = right;
170
171                 // 将整个加法表达式作为赋值语句的右子节点
172                 t->child[0] = p;
173             }
174         }
175         // 消耗掉标识符 token
176         match(ID);
177     }
178     return t;
179 }

```

图 4-2-12 新增自减符号运算文法规则匹配函数(同时实现了前置和后置自减)

```

82 //P394
83 //lineno: 961
84 TreeNode* statement(void)
85 {
86     TreeNode* t = NULL;
87     switch (token) {
88         case IF: t = if_stmt(); break;
89         case REPEAT: t = repeat_stmt(); break;
90         case ID: t = assign_stmt(); break;
91         case READ: t = read_stmt(); break;
92         case WRITE: t = write_stmt(); break;
93         case FOR: t = for_stmt(); break;
94         case WHILE: t = while_stmt(); break;
95         case SUBSUB: t = subsub_stmt(); break;
96         case PLUSPLUS: t = plusplus_stmt(); break;
97         default: syntaxError("unexpected token -> ");
98                 printToken(token, tokenString);
99                 token = getToken();
100                 break;
101     } /* end case */
102     return t;
103 }

```

图 4-2-13 修改后的 statement 函数

对于上面的 statement 函数，case 中 SUBSUB 和 PLUSPLUS 分别实现了前置自减和前置自增的实现，而额外添加的后置自减和后置自增则通过 assign_stmt 函数实现：

```
224  TreeNode* assign_stmt(void)
225  {
226      TreeNode* t = newStmtNode(AssignK);
227      if ((t) && (token == ID))
228          t->attr.name = copyString(tokenString);
229      match(ID);
230      if (token == ASSIGN)
231      {
232          match(ASSIGN);
233          if (t) t->child[0] = exp();
234      }
235      else if (token == PLUSEQ)
236      {
237          match(PLUSEQ);
238          //if (t) t->child[0] = exp();
239          if (t) {
240              TreeNode* p = newExpNode(OpK);
241              if (p != NULL) {
242                  TreeNode* temp = newExpNode(IdK);
243                  if (temp != NULL)
244                      temp->attr.name = copyString(t->attr.name);
245                  p->child[0] = temp;
246                  p->attr.op = PLUS;
247                  p->child[1] = exp();
248                  t->child[0] = p;
249              }
250          }
251      }
252      else if (token == REGEX)
253      {
254          match(REGEX);
255          if (t) t->child[0] = regex_stmt();
256      }
257      else if (token == PLUSPLUS)
258      {
259          match(PLUSPLUS);
260          if (t) {
261              TreeNode* p = newExpNode(OpK);
262              if (p != NULL) {
263                  TreeNode* temp = newExpNode(IdK);
264                  if (temp != NULL)
265                      temp->attr.name = copyString(t->attr.name);
266                  p->child[0] = temp;
267              }
268              p->attr.op = PLUS;
269              TreeNode* constNode = newExpNode(ConstK);
```

```

270         if (constNode != NULL) {
271             constNode->attr.val = 1;
272             p->child[1] = constNode;
273         }
274         t->child[0] = p;
275     }
276 }
277 else if (token == SUBSUB)
278 {
279     match(SUBSUB);
280     if(t) {
281         TreeNode* p = newExpNode(OpK);
282         if (p != NULL) {
283             TreeNode* temp = newExpNode(IdK);
284             if (temp != NULL)
285                 temp->attr.name = copyString(t->attr.name);
286             p->child[0] = temp;
287         }
288         p->attr.op = MINUS;
289         TreeNode* constNode = newExpNode(ConstK);
290         if (constNode != NULL) {
291             constNode->attr.val = 1;
292             p->child[1] = constNode;
293         }
294         t->child[0] = p;
295     }
296 }
297 return t;
298 }

```

图 4-2-14、图 4-2-15 修改后的 assign_stmt 函数

5. 系统测试

5-1 系统启动

由于已经生成了 Release 版本，直接双击可执行程序即可打开：



图 5-1-1 系统运行成功截图

5-2 数据测试

针对修改与扩充后的 TINY 语言语法规则，一共有 6 组测试，其测试样例与测试结果如下图所示：

测试 1:

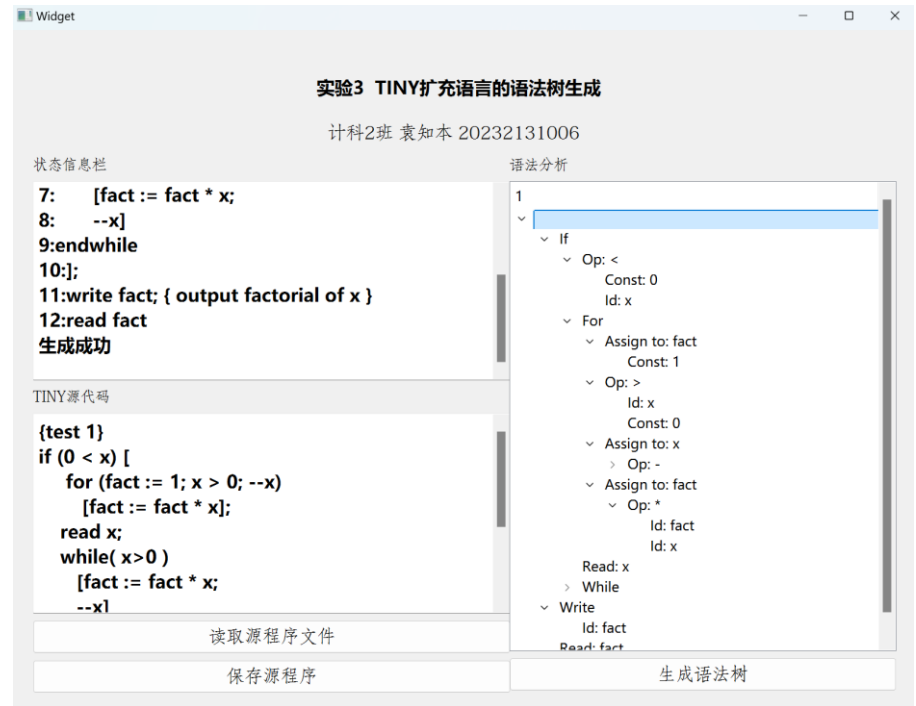


图 5-2-1 测试 1 成功

测试 2:

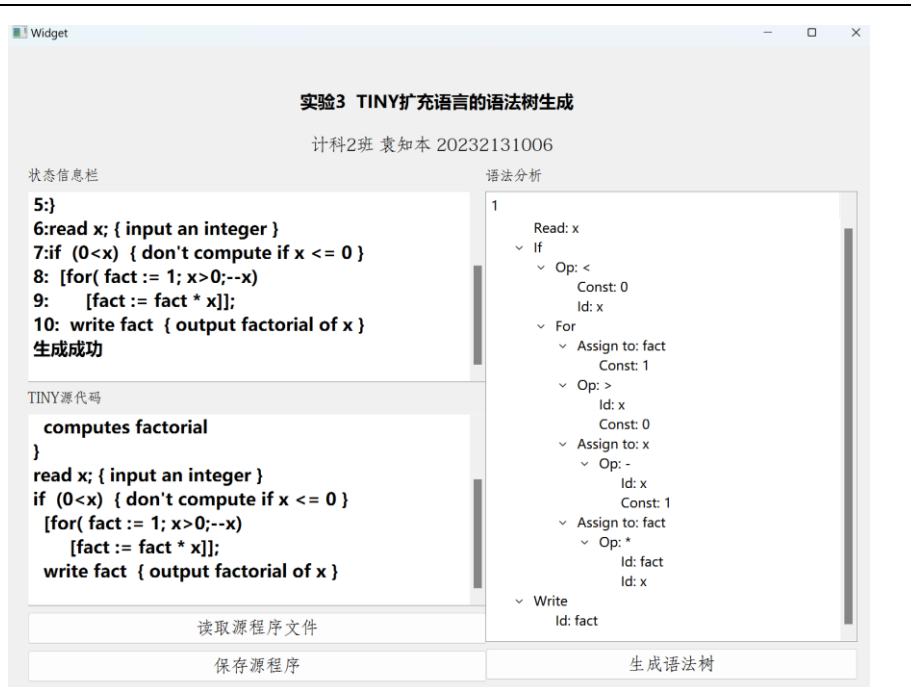


图 5-2-2 测试 2 成功

测试 3:



图 5-2-3 测试 3 成功

测试 4:

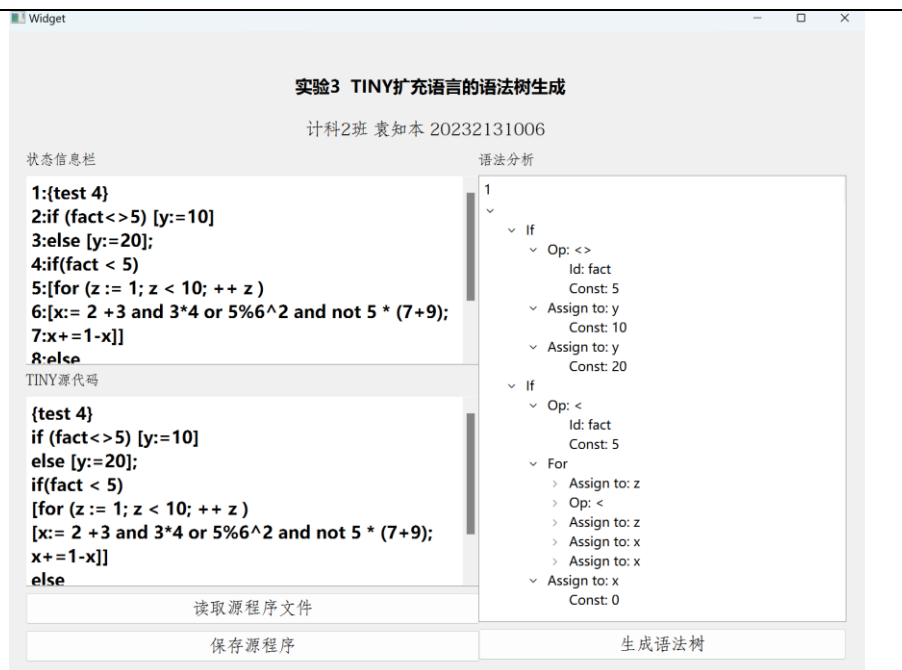


图 5-2-4 测试 4 成功

测试 5:



图 5-2-5 测试 5 成功

测试 6:

五、参考文献：

编译原理（3）——for 循环_编译原理实验三语法分析 for 循环-CSDN 博客：

<https://blog.csdn.net/zsh1184528359/article/details/109635198>

Tiny 扩充语言的语法树生成：

<https://blog.csdn.net/chaichai126/article/details/135319290>

编译原理 Tiny Syntax Tree：

<https://blog.csdn.net/phshp/article/details/139010754>